

# L25 — Infix to Postfix Conversion

Unit: 2 | Chapter: Stacks & Recursion | BT Level: BT5 | CO: CO3

---

## Learning Objectives

- Define operator precedence and associativity rules
  - Implement the Shunting-Yard algorithm for infix to postfix conversion
  - Trace the conversion for complex expressions with parentheses
  - Handle all standard operators including exponentiation
- 

## 1. Why Convert Infix to Postfix?

**Infix:**  $3 + 4 * 2 \rightarrow$  requires knowing precedence and parentheses

**Postfix:**  $3\ 4\ 2\ *\ + \rightarrow$  machine-evaluable left-to-right with a simple stack

Compilers convert infix arithmetic expressions to postfix (or prefix) internally before generating code. This is a core step in **expression parsing**.

---

## 2. Operator Precedence and Associativity

| Operator         | Precedence                 | Associativity |
|------------------|----------------------------|---------------|
| $^$ (exponent)   | 4 (highest)                | Right-to-Left |
| $*$ , $/$ , $\%$ | 3                          | Left-to-Right |
| $+$ , $-$        | 2                          | Left-to-Right |
| $($              | 1 (lowest when in stack) — |               |

### Rules:

- Higher precedence operators are applied first
  - Left-to-right:  $a - b - c = (a - b) - c$
  - Right-to-left:  $a ^ b ^ c = a ^ (b ^ c)$
- 

## 3. Shunting-Yard Algorithm (Dijkstra, 1961)

```
Algorithm infix_to_postfix(expression):  
    output = []           # Output queue (postfix result)  
    operator_stack = []  
  
    for each token in expression:  
        if token is a number/variable:  
            append to output  
  
            elif token is '(':
```

```

        push '(' onto operator_stack

    elif token is ')':
        while top of stack != '(':
            pop from stack and append to output
        pop '(' from stack (discard it)

    elif token is an operator:
        while (operator_stack not empty AND
              top != '(' AND
              (top has higher precedence than token OR
               (same precedence AND token is left-associative))):
            pop from stack and append to output
        push token onto operator_stack

while operator_stack not empty:
    pop from stack and append to output

return output

```

---

## 4. Implementation

```

def infix_to_postfix(expression):
    precedence = {'+': 2, '-': 2, '*': 3, '/': 3, '%': 3, '^': 4}
    right_assoc = {'^'}    # Right-associative operators

    output = []
    op_stack = []
    tokens = expression.split()

    for token in tokens:
        if token not in precedence and token not in '()':
            # Operand (number or variable)
            output.append(token)

        elif token == '(':
            op_stack.append(token)

        elif token == ')':
            while op_stack and op_stack[-1] != '(':
                output.append(op_stack.pop())
            if not op_stack:
                raise ValueError("Mismatched parentheses")
            op_stack.pop()    # Remove '('

        else:
            # Operator
            while (op_stack and
                  op_stack[-1] != '(' and
                  op_stack[-1] in precedence and
                  (precedence[op_stack[-1]] > precedence[token] or

```

```

        (precedence[op_stack[-1]] == precedence[token] and
         token not in right_assoc)):
            output.append(op_stack.pop())
            op_stack.append(token)

while op_stack:
    if op_stack[-1] == '(':
        raise ValueError("Mismatched parentheses")
    output.append(op_stack.pop())

return ' '.join(output)

```

---

## 5. Detailed Traces

### Example 1: $A + B * C$

| Token | Action                                   | Stack  | Output          |
|-------|------------------------------------------|--------|-----------------|
| A     | Operand                                  | []     | [A]             |
| +     | Push (empty stack)                       | [+]    | [A]             |
| B     | Operand                                  | [+]    | [A, B]          |
| *     | $\text{prec}(*) > \text{prec}(+)$ , push | [+, *] | [A, B]          |
| C     | Operand                                  | [+, *] | [A, B, C]       |
| END   | Pop all: *, then +                       | []     | [A, B, C, *, +] |

**Result:**  $A \ B \ C \ * \ +$

**Verify:** evaluate  $A \ B \ C \ * \ + \rightarrow A + (B * C) = A + B * C \checkmark$

---

### Example 2: $(A + B) * C$

| Token | Action               | Stack  | Output          |
|-------|----------------------|--------|-----------------|
| (     | Push                 | [(]    | []              |
| A     | Operand              | [(]    | [A]             |
| +     | Push (top is '(')    | [(, +] | [A]             |
| B     | Operand              | [(, +] | [A, B]          |
| )     | Pop until '(': pop + | []     | [A, B, +]       |
| *     | Push                 | [*]    | [A, B, +]       |
| C     | Operand              | [*]    | [A, B, +, C]    |
| END   | Pop *:               | []     | [A, B, +, C, *] |

**Result:**  $A \ B \ + \ C \ *$

**Verify:**  $(A + B) * C \checkmark$

---

### Example 3: $A \wedge B \wedge C$ (right-associative)

| Token | Action                                   | Stack  | Output          |
|-------|------------------------------------------|--------|-----------------|
| A     | Operand                                  | []     | [A]             |
| ^     | Push                                     | [^]    | [A]             |
| B     | Operand                                  | [^]    | [A, B]          |
| ^     | Right-assoc: same prec, don't pop → push | [^, ^] | [A, B]          |
| C     | Operand                                  | [^, ^] | [A, B, C]       |
| END   | Pop ^, ^:                                | []     | [A, B, C, ^, ^] |

**Result:** A B C ^ ^

**Verify:** A ^ (B ^ C) (right-to-left associativity) ✓

### Example 4: 3 + 4 \* 2 / ( 1 - 5 ) ^ 2 ^ 3

| Token | Stack                       | Output                      |
|-------|-----------------------------|-----------------------------|
| 3     | []                          | [3]                         |
| +     | [+]                         | [3]                         |
| 4     | [+]                         | [3,4]                       |
| *     | [+,*]                       | [3,4]                       |
| 2     | [+,*]                       | [3,4,2]                     |
| /     | pop * → [+], push / → [+,/] | [3,4,2,*]                   |
| (     | [+,/, (]                    | [3,4,2,*]                   |
| 1     | [+,/, (]                    | [3,4,2,*,1]                 |
| -     | [+,/, (-]                   | [3,4,2,*,1]                 |
| 5     | [+,/, (-]                   | [3,4,2,*,1,5]               |
| )     | pop -: [+,/]                | [3,4,2,*,1,5,-]             |
| ^     | [+,/, ^]                    | [3,4,2,*,1,5,-]             |
| 2     | [+,/, ^]                    | [3,4,2,*,1,5,-,2]           |
| ^     | right-assoc: [+,/, ^, ^]    | [3,4,2,*,1,5,-,2]           |
| 3     | [+,/, ^, ^]                 | [3,4,2,*,1,5,-,2,3]         |
| END   | pop ^, ^, /, +              | [3,4,2,*,1,5,-,2,3,^,^,/,+] |

**Result:** 3 4 2 \* 1 5 - 2 3 ^ ^ / +

## 6. Complexity

| Metric | Value                                                  |
|--------|--------------------------------------------------------|
| Time   | O(n) — each token processed at most twice (push + pop) |
| Space  | O(n) — stack holds at most n/2 operators               |

## Summary

- Infix  $\rightarrow$  Postfix conversion uses a stack to handle precedence and parentheses.
  - **Shunting-Yard algorithm:** push operands directly, pop operators with higher/equal precedence before pushing current.
  - Parentheses ( and ) control grouping — they never appear in postfix output.
  - Right-associative operators (like  $\wedge$ ) are not popped when equal precedence matches.
  - Time and space:  $O(n)$ .
- 

## References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.1 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.4 (Springer, 2020)
- Dijkstra, "Algol 60 Translation" (1961) — original Shunting-Yard paper